

04

Deploy day

Environment variables, databases, domains, SSL, secrets. The pre-flight list, in the order things bite you.

SUBJECT	PRODUCTION • ENVIRONMENTS
AUTHOR	MICAH JONES
TIME	A TEN-MINUTE READ

READER	SOLO BUILDERS ON AI TOOLS
STATUS	CHAPTER FOUR OF TEN
REV	2026.08

Production is a different machine

Chapter one told you about my three lead forms that said “Got it” to every visitor for weeks while delivering nothing, because one environment variable never made it to the live host. This chapter is that story’s autopsy, generalized: why the demo lies, and the exact order in which production bites.

The mental model that explains nearly every deploy-day failure fits in one sentence: your laptop is full of invisible help, and production has none of it.

YOUR LAPTOP

```
.env file with every key
logged-in CLIs
database you seeded by
hand
localhost URLs everywhere
files from six months of
work
```

PRODUCTION

```
your code

...and nothing else.
Every value you didn't
explicitly provide is
missing.
```

Six months of building leaves your machine soaked in implicit state: keys in a local file, tools already logged in, a database you seeded by hand in week two and forgot about, URLs that say localhost. Your app doesn’t just run your code. It runs your code plus all of that, and only the code makes the trip to production.

Deploy day is one job: making every piece of invisible help explicit, on a machine that starts with nothing.

FIELD NOTE

This is also why “works on my machine” is a punchline among engineers. It is always true and never useful. The machine that matters is the one strangers reach.

The order things bite you

1. Environment variables

The number-one killer, and the reason chapter one’s forms died. Your code reads keys and URLs from the environment. Locally they live in a `.env` file that never leaves your machine, which is correct. On the host, every one of them must be entered by hand, and there are usually three separate environments to enter them into: production, previews, and development.

Two traps inside the trap. First, the silent kind of missing: a well-meaning AI wraps the key check in a fallback, so instead

of crashing, the app logs a warning nobody reads and tells the user everything worked. Prefer the loud failure. A crash on deploy day beats a lie for a month.

FROM THE BUILD LOG

ENTRY • 2026-08-31

Installed is not live

The morning I fixed the dead forms, I added the missing email key to the host, re-ran my test, and it was still dead. Ten minutes of confusion, then the second lesson of the day: environment variables are read when the app is built, not when you save them in the dashboard.

Nothing changes until the next deploy.

The rule that stuck: env change, then redeploy, then test. Three steps, always in that order, never two.

The habit that makes this box safe: keep a committed `.env.example`, the ledger of every variable your code reads, with no values. It is the invariant list from chapter one, wearing an ops badge.

`.env.example`

```
# Every var the code reads. No values here, ever.
# Sends all transactional email. Sending-only key.
RESEND_API_KEY=
# Postgres connection. Prod value lives ONLY on the
host.
DATABASE_URL=
# Stripe. Webhook secret pairs with the endpoint,
ch. 6.
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
```

2. The database

Your production database is born empty, and it is not the one on your laptop. Three things bite here, in order: the connection string is an environment variable (see above, twice); your schema must arrive by migrations, not by the pile of hand-edits your local database accumulated; and whatever safety rules you built in chapter three, row-level security included, must actually be enabled in the hosted

instance. “It worked locally” often means “my local database was lawless and forgiving.”

3. Domains, SSL, redirects

Your app is live on the platform’s URL in minutes. The day feels done. Then you wire the real domain, and this box turns out to have teeth: DNS records, certificates, and the apex-versus-www question every site answers whether it knows it or not.

The rule: one of the two is the real address, the other one redirects to it, exactly one hop. Then verify all three doors: the apex, the www, and the platform URL, all serving the same build.

FROM THE BUILD LOG

ENTRY • 2026-08-31

The redirect loop I shipped this afternoon

While writing this very chapter, I made the classic move. My www domain was manually pinned to a specific deployment, which meant every deploy required me to re-point it by hand, and forgetting would serve visitors last week’s site. Fixing that properly meant attaching www to the project. I ran the command.

The platform attached www with a default redirect to the apex. The apex already redirected to www. Both public doors spun in a redirect loop, fifty hops deep, and my site was down for two minutes on a Sunday while I unwound it.

Two lessons. Redirects have two ends, and you check both directions before and after touching either. And a two-minute outage you catch yourself, on a quiet day, is the cheap tuition: the check I now run took thirty seconds to write and runs every time.

4. Public URLs and callbacks

Localhost is a private address. The moment a third party needs to reach you (a payment webhook, an email bounce, a calendar callback), your app needs a public URL, and every one of those integrations you tested locally was tested against a machine the outside world cannot see. Each has a live-mode switch and a callback URL field, and each must be

re-pointed at the real domain. Chapter 6 walks the payment version in detail, because that is the one that costs money.

5. The frozen client

One subtlety that bites late: some of your configuration is baked into the browser bundle at build time. In most frameworks, that means the variables with a public prefix. Change them and, as with every environment variable, the change means nothing until a rebuild. If a “fixed” value keeps showing up wrong, you are usually looking at a stale build, not a stubborn bug.

Verify in final form

The through-line of all five boxes, and of chapter one’s dead forms: “it works” is a claim about the exact path you tested, on the machine that matters. Testing the form on localhost proves localhost. The only test that counts is the one a stranger could have run: the live site, the real form, the actual email arriving in an inbox you can open.

So deploy day ends with a ritual, not a feeling:

§ 04.3

- ☐ **List every variable the code reads**, and confirm each exists on the host for each environment.
Your `.env.example` is the checklist; the AI can generate it by searching the code for every environment read.

- ☐ **Env change → redeploy → test.** In that order, every time. Installed is not live until the next build.

- ☐ **Migrate the hosted database and confirm its safety rules are on.** Your laptop's database is not evidence.

- ☐ **Open all three doors:** apex, www, and the platform URL. Confirm one redirect hop at most and the same build behind each.

- ☐ **Fire every integration once, for real, on the live site.**
Submit each form and watch the artifact arrive: the email in the inbox, the row in the database, the webhook in the log. Ten minutes that would have saved my forms three weeks of silence.

The app is live, the doors all open, the machinery verified from the outside. Which is exactly when a different question arrives: now that strangers can reach it, what can the wrong stranger do? That is the next chapter.

The security pre-flight

Row-level security done right, the auth pattern that survives, and the hardcoded keys you left in. Two checks catch most of it.